

— AI & DATA SCIENCE TRACK · ROUND 2 · NHA-4-190

AI-Powered Predictive Maintenance for Industrial Equipment

Machine Learning + IoT Sensor Data to forecast equipment failures before they occur — enabling just-in-time maintenance at industrial scale.

Python

scikit-learn

XGBoost

FastAPI

SMOTE

PSI Monitoring

Uvicorn

- System Deployed · Real-time inference API running

— THE PROBLEM

Industrial Equipment Failures Are Costing Billions

\$50B

Annual cost of unplanned downtime worldwide

30–50%

Of all maintenance is still reactive — fix-when-broken

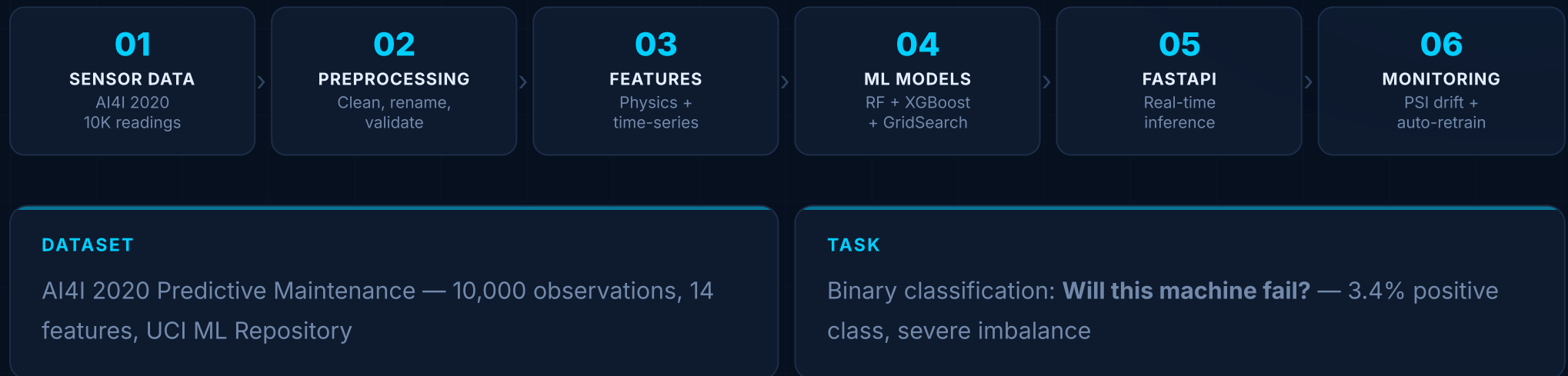
70%

Of equipment failures are preventable with early warning

- **Reactive maintenance** causes cascading damage, safety hazards, and high emergency repair costs
- **Scheduled maintenance** wastes labor and parts by servicing equipment that doesn't need it
- **Predictive maintenance** uses real sensor readings to intervene at exactly the right time

— THE SOLUTION

End-to-End ML Pipeline for Real-Time Failure Prediction



— MILESTONE 1 & 2

Data Preprocessing & Feature Engineering

RAW INPUTS — 14 COLUMNS

Air Temperature	Kelvin — ambient
Process Temperature	Kelvin — internal
Rotational Speed	RPM — spindle
Torque	Nm — applied force
Tool Wear	Minutes — cumulative
Type + 5 Failure Flags	L/M/H · TWF HDF PWF OSF RNF

ENGINEERED FEATURES — 6 ADDED

Temp_Diff	Process – Air → thermal stress
Power	Torque × RPM → mechanical load
Temp_Rate_of_Change	pct_change — rapid shifts
Torque_Rate_of_Change	pct_change — stress spikes
Rolling_Avg_Temp	window=5 — smoothed trend
Rolling_Std_Torque	window=5 — instability

Failure rate: ~3.4% — severe class imbalance addressed with **SMOTE applied to training data only** (no leakage into test set). Train/test split 80/20, `shuffle=False` to preserve temporal order.

— MILESTONE 3

Model Development & Optimization

MODEL	STRATEGY	F ₂ THRESHOLD	STATUS
Random Forest — Baseline	100 trees, default params	0.45	Trained
Random Forest — Optimized	GridSearch: trees, depth, splits · 3-fold CV	0.48	Deployed
XGBoost — Baseline	Default XGBClassifier, logloss	0.81	Trained
XGBoost — Optimized	GridSearch: depth, LR, n_estimators	0.47	Trained

WHY FB WITH B=2?

Missing a failure costs far more than a false alarm. F₂ weights recall **twice as heavily** as precision — the correct trade-off for industrial safety.

WHY RANDOM FOREST?

Best recall at the optimized threshold, robust cross-validation, and **interpretable feature importances** for maintenance teams.

RESULTS

Key Findings & Insights

TOP PREDICTORS OF FAILURE



DEGRADATION PATTERN

Temperature trends **upward** and torque variance **spikes** in the 20–50 steps before failure — captured by rolling features

PCA ANALYSIS

2-component PCA shows **partial but real separation** between normal and failure clusters — confirming engineered features carry discriminative signal

0.48

Optimal threshold

$\beta=2$

Recall-weighted scoring

3.4%

Failure rate (imbalanced)

16

Total engineered features

— MILESTONE 4

Deployment & Real-Time Inference

FASTAPI SERVICE — 5 ENDPOINTS

GET /health Liveness + model metadata

POST /predict Real-time failure prediction

POST /feedback Submit true outcome labels

GET /metrics Volume & alert rate

GET /drift PSI drift detection

ONLINE INFERENCE FLOW

- Sensor payload arrives at **POST /predict**
- Physics features (**Temp_Diff** , **Power**) computed inline
- **RollingFeatureStore** generates time-series features per asset ID
- Features aligned to **model.feature_names_in_** — column order guaranteed
- Probability vs threshold (0.48) → binary decision returned and logged to CSV

— MILESTONE 4 — MONITORING

Continuous Monitoring & Auto-Retraining

DATA DRIFT — PSI

Population Stability Index between training distributions and live prediction logs, per feature.

Alert threshold: $\text{PSI} > 0.20$

ONLINE PERFORMANCE

Precision, Recall, F1, ROC-AUC computed from feedback labels on live samples.

Recall threshold: > 0.80

AUTO-RETRAINING

When drift or recall breach detected, full pipeline re-executes automatically via `python main.py`.

PREDICTION LOG

`predictions_log.csv` — timestamp, request_id, asset_id, probability, decision, all feature values

REPRODUCIBILITY

All config in `config/config.yaml` · deterministic seeds · `python main.py` reruns the full pipeline

— BUSINESS IMPACT

Why This Matters in Production

30–50%

Reduction in unplanned downtime

20–40%

Extension of equipment lifespan

COST REDUCTION

Eliminates emergency repairs, reduces spare parts inventory, and cuts overtime labor costs from reactive crises.

SMART SCHEDULING

Shifts from fixed-interval to condition-based maintenance — service only when sensor data says it's needed.

FLEET VISIBILITY

Real-time API provides continuous, per-machine health monitoring across entire production fleets.

— ENGINEERING DECISIONS

Challenges & How We Solved Them

Severe class imbalance — only 3.4% failures

SMOTE applied to training set only + F_2 threshold optimization ($\beta=2$ penalizes missed failures 2× more than false alarms)

Temporal data leakage risk

`shuffle=False` preserves time ordering; SMOTE applied after splitting to prevent test set contamination

Online feature parity — training vs. production

`RollingFeatureStore` in `serving.py` replicates rolling-window logic per asset ID, mirroring offline feature engineering exactly

Model drift in production

PSI-based monitoring detects distribution shift; automated retraining re-executes when $PSI > 0.20$ or $recall < 0.80$

Reproducibility across environments

Central `config/config.yaml` as single source of truth; deterministic random seeds; single-command pipeline

— LOOKING AHEAD

Future Work & Extensions

01

LSTM / Transformer Models

Capture longer-range temporal dependencies in sensor sequences that tree models miss

02

Multi-Class Failure Prediction

Predict specific failure type — TWF, HDF, PWF, OSF, RNF — not just binary fail/no-fail

03

Remaining Useful Life (RUL)

Predict not just *if* but *when* — enabling precision maintenance scheduling

04

Edge Deployment

Compile model for IoT edge devices co-located with equipment — zero network latency

05

Web Dashboard

Real-time fleet health dashboard for maintenance teams with alerts and trend visualizations

06

A/B Testing Framework

Validate retraining improvements with controlled production experiments

Thank You

A complete, production-ready predictive maintenance system — from raw IoT sensor data to a deployed real-time inference API with continuous monitoring and automated retraining.

Python

pandas

scikit-learn

XGBoost

imbalanced-learn

FastAPI

Uvicorn

matplotlib

seaborn

PyYAML

joblib

Navigate with arrow keys or the buttons below